

# Como predecir si alguien pagara su préstamo con ML

---

Clasificación de Riesgo Crediticio usando ML

## Gary Briceño

- Ingeniero Electronico
- Universidad Nacional de Ingeniería - Perú
- Estudiante de la Maestría de IA
- Universidad de los Andes - Colombia

### Enlaces:

- <https://www.linkedin.com/in/garybriceno/>
- <https://github.com/GaryBriceno>



# Agenda

---

1. Introduction
2. Contexto
3. Presentación y procesamiento del dataset
4. Modelado con árbol de decisión
5. Evaluación del modelo
6. Optimizacion
7. Conclusiones

# Predicción de Préstamo usando ML

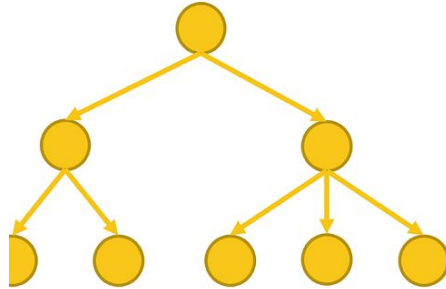
---

- En este caso, vamos a evaluar la información que tenemos para en base a ella, y utilizando **arboles de clasificacion**, poder predecir si a futuros clientes se les otorga o no un prestamo bancario, en base a las características de la información.
- Es como ir deduciendo la respuesta en base a preguntas:
  - El cliente tiene más de 30 años?
  - El crédito solicitado es mayor a 5000?
  - Tiene cuenta de ahorros?
  - Que tipo de propiedad tiene?
- Cada pregunta nos llevará más cerca a una predicción.

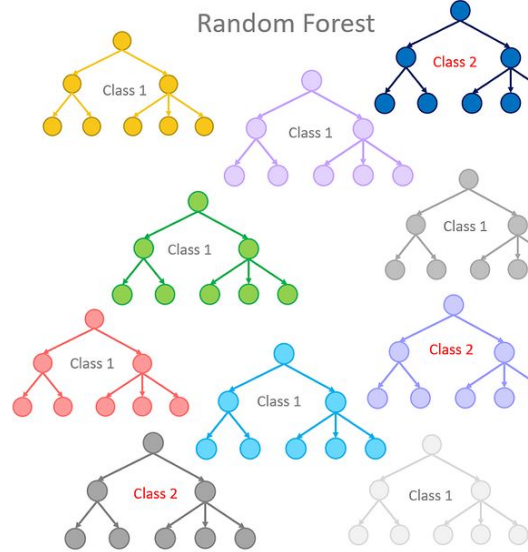
# Introduccion

- Vamos a realizar la clasificación utilizando algunos conceptos como
  - Arboles de Decision
  - Random Forest

Single Decision Tree

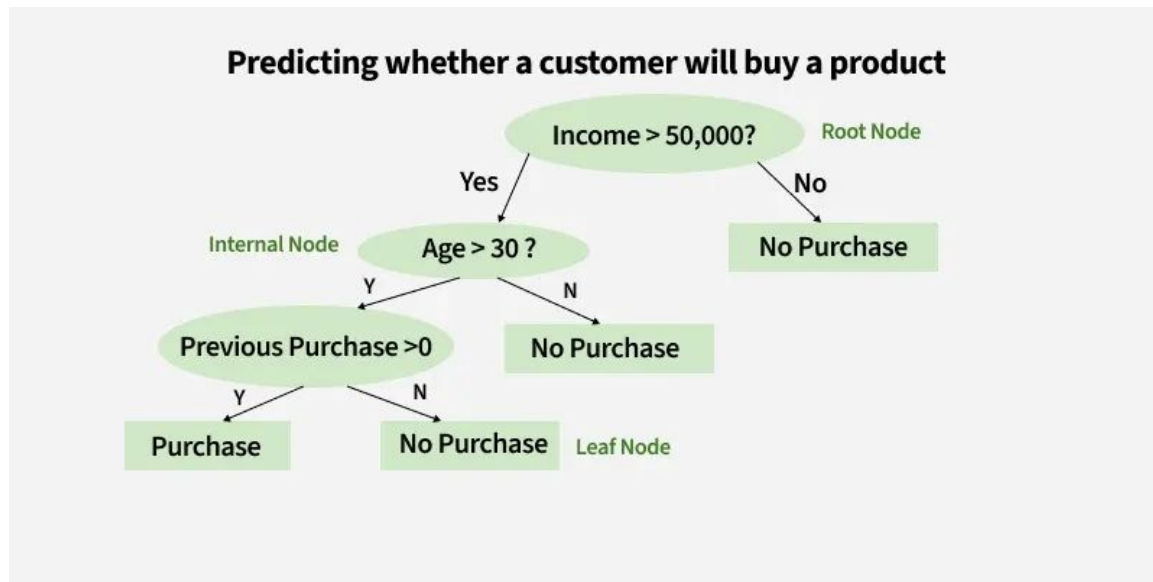


Random Forest



# Que es un árbol de Decisión?

- Es un **modelo supervisado**, que predice una variable objetivo dividiendo espacios de decisiones en una estructura jerárquica de condiciones.



# Arbol de Decision

---

- Ventajas

- Facil de entender e interpretar
- Acepta variables numéricas y categóricas
- No requiere normalización de datos

- Desventajas

- Tiene al sobreajuste (overfitting)
- Sensible a pequeños cambios de datos

# Que es un Random Forest?

---

- Es cuando dentro de los datos, se pueden identificar **varios** árboles de decisión.
- Es un modelo más robusto y preciso.
- Como funciona?
  - Crea múltiples árboles con subconjuntos aleatorios del dataset
  - Cada árbol tiene un peso, y se toma la mayoría como predicción final



# Random Forest

---

- Ventajas

- Más preciso que un solo árbol
- Menos propenso al overfitting
- Maneja grandes datasets con mucho mas variables

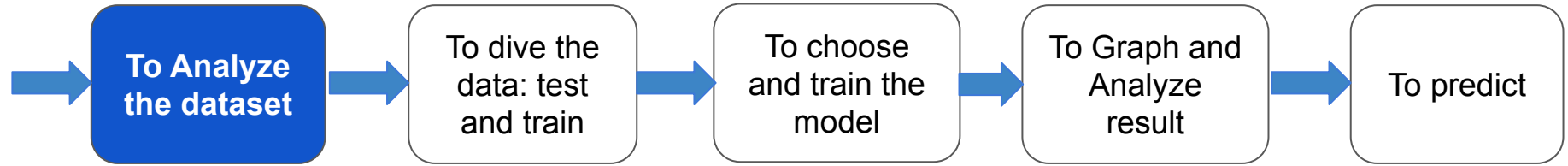
- Desventajas

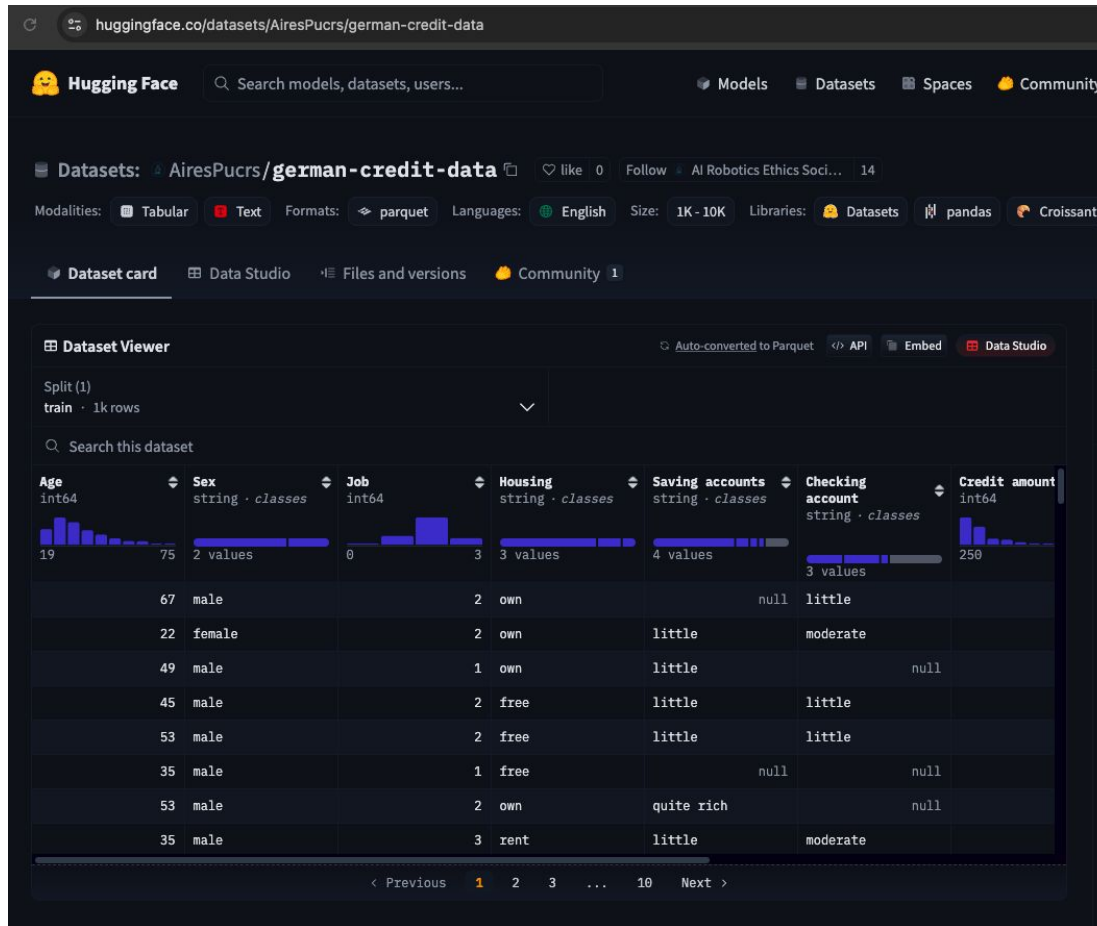
- Mas lento de entrenar
- Menos interpretable que un solo árbol

# Dataset

# Process

---

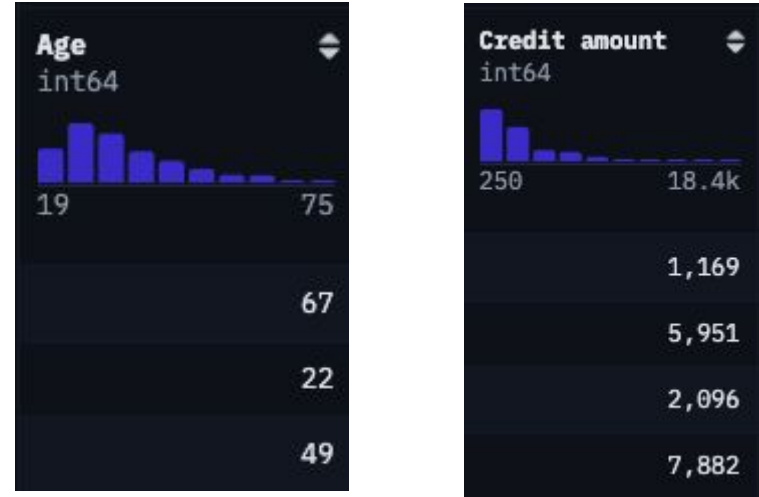




# Variables Categoricalas



# Variables Numericas



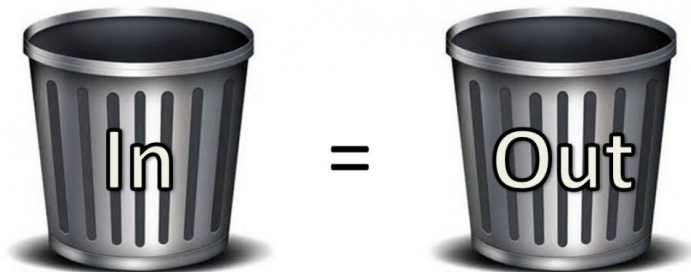
# Datos incompletos

---

|   | Age | Sex    | Job | Housing | Saving accounts | Checking account | Credit amount | Duration | Purpose             | Risk |
|---|-----|--------|-----|---------|-----------------|------------------|---------------|----------|---------------------|------|
| 0 | 67  | male   | 2   | own     | None            | little           | 1169          | 6        | radio/TV            | good |
| 1 | 22  | female | 2   | own     | little          | moderate         | 5951          | 48       | radio/TV            | bad  |
| 2 | 49  | male   | 1   | own     | little          | None             | 2096          | 12       | education           | good |
| 3 | 45  | male   | 2   | free    | little          | little           | 7882          | 42       | furniture/equipment | good |
| 4 | 53  | male   | 2   | free    | little          | little           | 4870          | 24       | car                 | bad  |

# Análisis de Datos

---

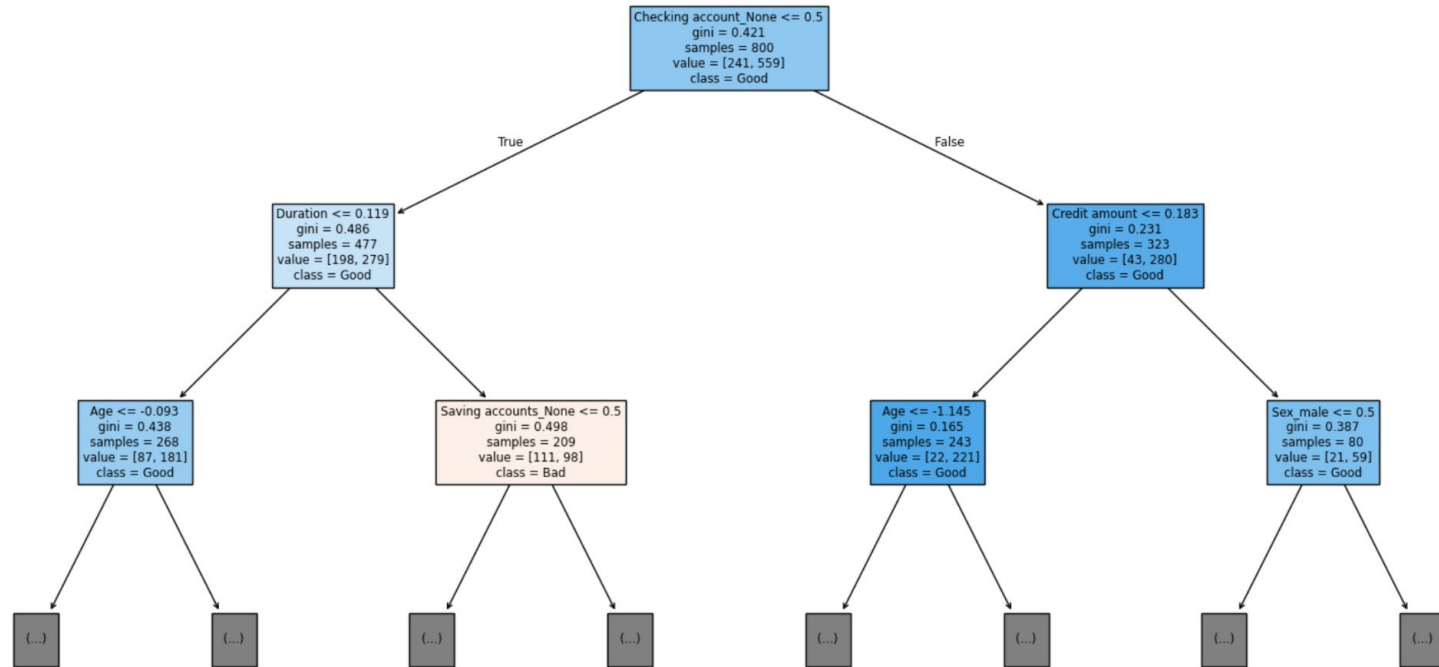


- Calidad de los datos:
  - Valores faltantes, valores extremos, datos correctos
- Distribución de las variables
  - Se requieren escalar (Feature Engineering), revisar si hay sesgos, revisar las correlaciones
- Balance de las clases
  - Hay más instancias de una clase, que de otra.

# Arbol de Decision



# Arbol de Decision



Github: [https://github.com/GaryBriceno/ml\\_examples/blob/main/beginners/03\\_clasificacion\\_riesgo\\_credificio.ipynb](https://github.com/GaryBriceno/ml_examples/blob/main/beginners/03_clasificacion_riesgo_credificio.ipynb)

# Optimizacion

# Optimizacion

---

```
# Optimización con GridSearch  
# En este caso, se realizan todas las combinaciones posibles de las variables de param_grid  
# Con cada combinacion, se entrena y evalua el modelo  
# Nos indica la mejor combinacion de parametros  
  
param_grid = {  
    'clf__max_depth': [3,5,7],  
    'clf__min_samples_split': [2,5,10],  
    'clf__criterion': ['gini','entropy']  
}  
  
grid = GridSearchCV(pipeline, param_grid, cv=5, scoring='accuracy')  
grid.fit(X_train, y_train)  
print("Mejores params:", grid.best_params_, "→ CV score:", grid.best_score_)
```

# Gracias